

ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT REPORT

Exploratory Data Analysis, Statistical Analysis and Machine Learning using the Titanic Dataset

Student Name: [Enter your name]

Index Number: [Enter your index number]

Programme: [Enter your programme]

Lecturer: Dr. Joseph Dadzie

Academic Year: 2025/2026, Second Semester

Date of Submission: [Enter date]

Table of Contents

1. Introduction.....	3
2. Dataset Description.....	3
3. Methodology.....	3
3.1 Data Acquisition.....	3
3.2 Data Cleaning.....	3
3.3 Data Visualisation.....	3
3.4 Statistical Analysis.....	4
3.5 Machine Learning.....	4
4. Results.....	4
4.1 Data Overview.....	4
4.2 Visualisations.....	4
4.3 Statistical Findings.....	7
4.4 Machine Learning Model Performance.....	7
5. Discussion.....	8
6. Conclusion.....	8
7. References.....	9
8. Appendix: Python Code.....	9

1. Introduction

The sinking of the RMS Titanic on 15 April 1912 remains one of the most widely studied maritime disasters in history, both for its human toll and for the rich passenger records it left behind. This project uses the Titanic passenger dataset, made available through Kaggle's "Titanic: Machine Learning from Disaster" competition, to demonstrate practical data science skills using the Python programming language.

The objectives of this project are to: (i) acquire and inspect a real-world dataset; (ii) clean and preprocess the data to handle missing values and duplicates; (iii) visualise key variables and relationships; (iv) carry out descriptive and correlation-based statistical analysis; and (v) build a supervised machine learning model to predict passenger survival. The tools used include Python, Jupyter Notebook, Pandas, NumPy, Matplotlib, Seaborn, and Scikit-Learn, in line with the software requirements specified for the BCS 404 course project.

2. Dataset Description

The dataset used is the Titanic training dataset (train.csv), obtained from Kaggle at <https://www.kaggle.com/competitions/titanic/data>. It contains 891 records, each describing one passenger aboard the Titanic, across 12 original variables.

Variable	Description
PassengerId	Unique identifier for each passenger
Survived	Survival indicator (0 = No, 1 = Yes) — target variable
Pclass	Ticket / passenger class (1 = 1st, 2 = 2nd, 3 = 3rd)
Name	Passenger's full name
Sex	Passenger's sex (male / female)
Age	Passenger's age in years
SibSp	Number of siblings / spouses aboard
Parch	Number of parents / children aboard
Ticket	Ticket number
Fare	Passenger fare
Cabin	Cabin number
Embarked	Port of embarkation (C, Q, S)

The variable of primary interest is Survived, which is used as the target label for the classification model developed in Section 6.

3. Methodology

3.1 Data Acquisition

The dataset was loaded into a Pandas DataFrame using `pandas.read_csv()`. Its dimensions, column names, first five records, and data types were inspected to confirm successful loading and to understand the structure of the data.

3.2 Data Cleaning

Missing values were detected using `DataFrame.isnull().sum()`. Three columns had missing values: Cabin ($\approx 77\%$ missing), Age ($\approx 20\%$ missing), and Embarked (2 missing values). Age was imputed using the median age within each combination of passenger class and sex, since age varies systematically with both variables. Cabin was dropped entirely due to its high proportion of missing data. Embarked was filled with its mode, since only two records were affected. The dataset was also checked for duplicate rows using `DataFrame.duplicated()`; none were found.

3.3 Data Visualisation

Six visualisations were produced using Matplotlib and Seaborn: a histogram of passenger ages, a bar chart of passenger class distribution, a boxplot of age by passenger class, a scatter plot of age versus fare, a correlation heatmap of numerical variables, and a pairplot of selected numerical variables. Each figure was labelled with a title and axis labels, and interpreted in relation to the research objectives.

3.4 Statistical Analysis

Descriptive statistics (mean, standard deviation, quartiles, etc.) were computed for all variables. Frequency distributions were produced for the key categorical variables (Survived, Pclass, Sex, Embarked). A Pearson correlation matrix was computed for the numerical variables to identify the strongest positive and negative relationships.

3.5 Machine Learning

A binary classification model was built to predict passenger survival using Logistic Regression from Scikit-Learn. The predictor variables used were Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked (one-hot encoded). The Sex variable was label-encoded (male = 0, female = 1). The data was split into training (80%) and testing (20%) subsets using a fixed random seed (random_state = 42) with stratification on the target variable to preserve class balance. The model was trained on the training set and evaluated on the held-out test set using accuracy, a confusion matrix, and a classification report.

4. Results

4.1 Data Overview

The cleaned dataset consists of 891 records across 11 columns after removing the Cabin column. No duplicate rows were found in the original data.

4.2 Visualisations

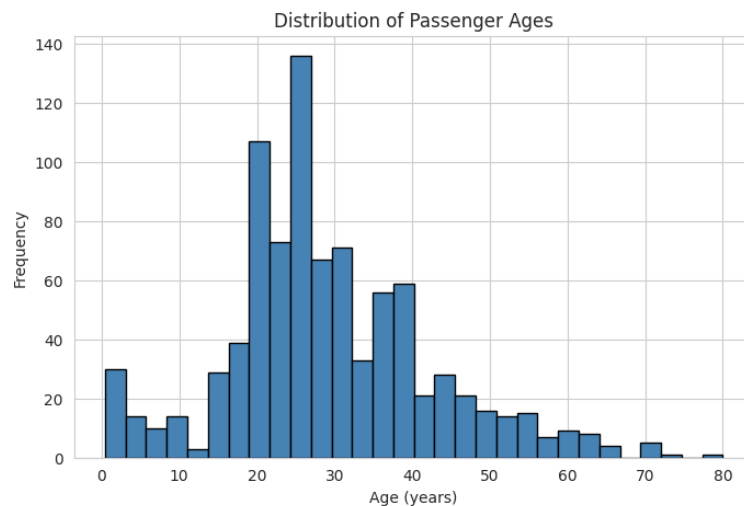


Figure 1: Distribution of Passenger Ages

Passenger ages are right-skewed, with most passengers between roughly 20 and 40 years old and a smaller secondary group of young children.

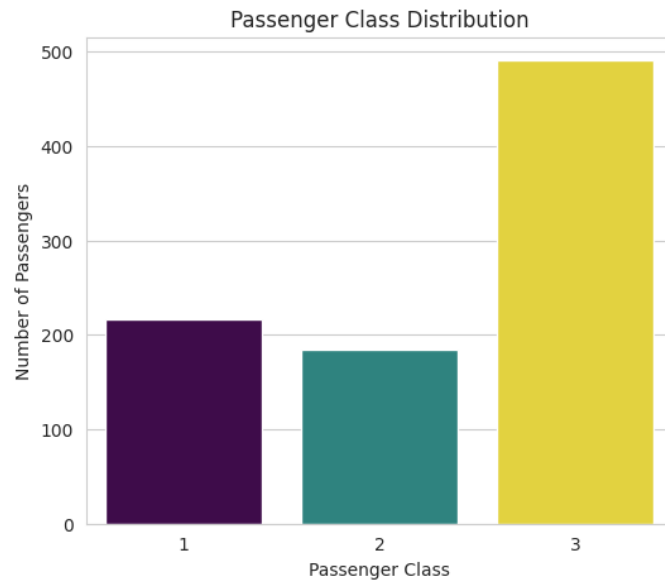


Figure 2: Passenger Class Distribution

Third class passengers form the largest group aboard, followed by first class and then second class.

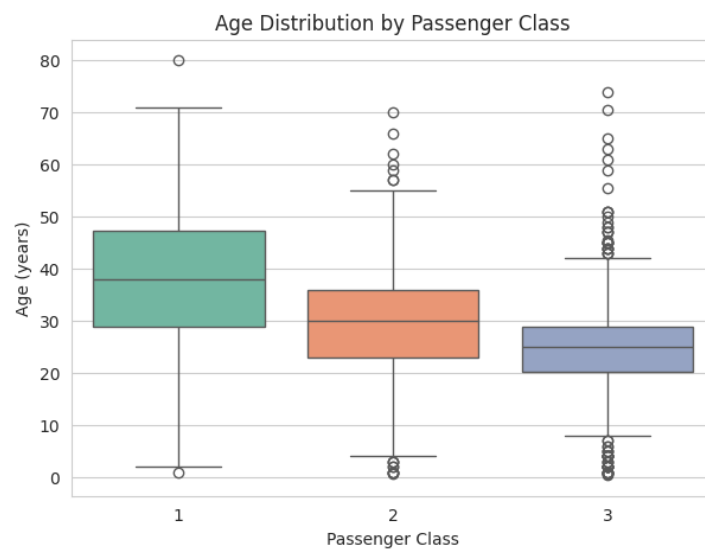


Figure 3: Age Distribution by Passenger Class

Median passenger age decreases from first class to third class, indicating that wealthier passengers in first class tended to be older.

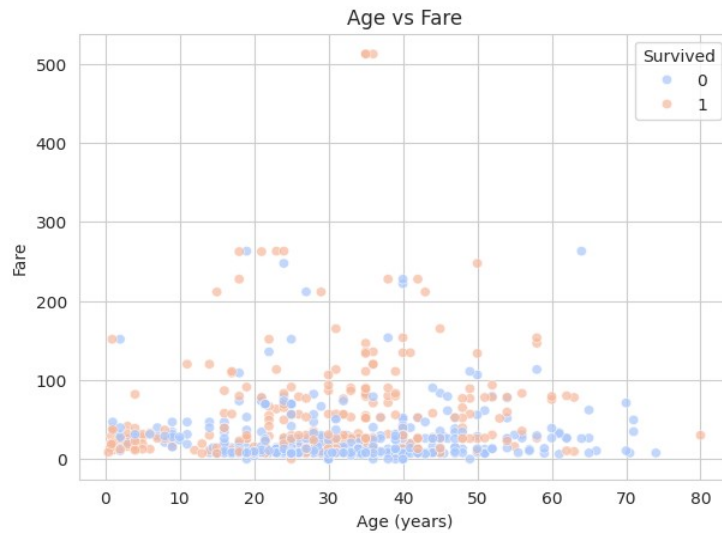


Figure 4: Age versus Fare, Coloured by Survival

Most fares cluster below 100, with a few high-fare outliers. Passengers who paid higher fares show a visibly higher concentration of survivors.

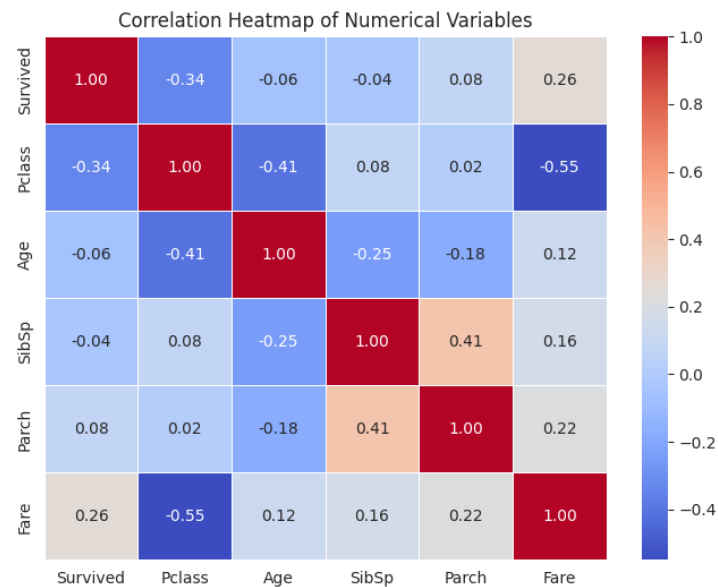


Figure 5: Correlation Heatmap of Numerical Variables

Pclass and Fare show a strong negative correlation, while Fare and Survived show a positive correlation, suggesting that wealthier, higher-fare passengers were more likely to survive.

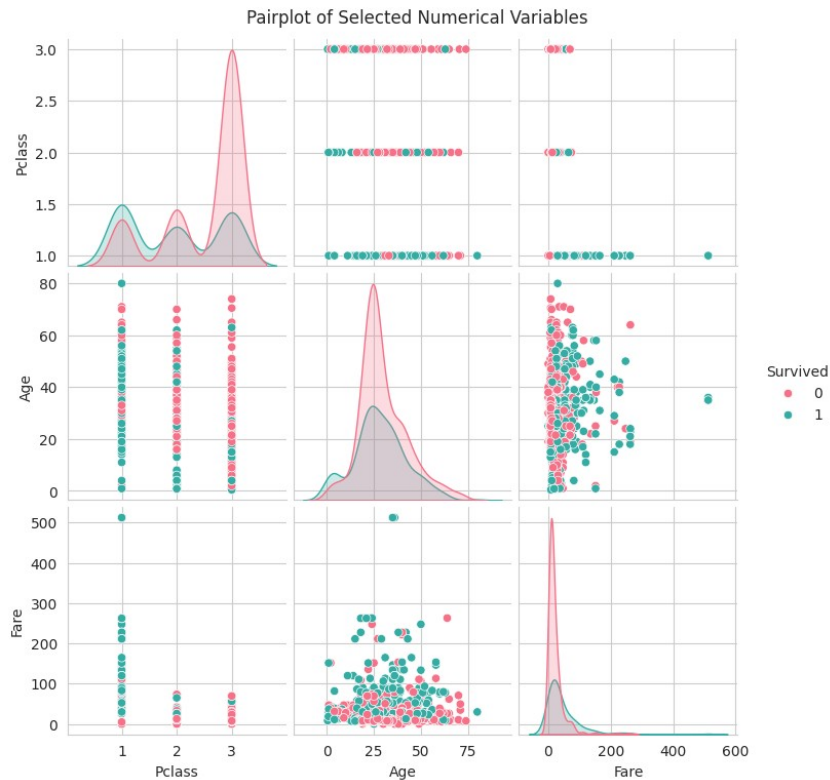


Figure 6: Pairplot of Survived, Pclass, Age and Fare

Survivors are visibly concentrated in lower Pclass values and higher Fare values compared to non-survivors, while the Age distributions of the two groups overlap substantially.

4.3 Statistical Findings

The strongest positive correlation among numerical variables was between SibSp and Parch ($r \approx 0.41$), reflecting passengers travelling together as family units. The strongest negative correlation was between Pclass and Fare ($r \approx -0.55$), confirming that higher-numbered (lower-status) classes paid substantially lower fares.

Group	Survival Rate
Female passengers	74.2%
Male passengers	18.9%
1st Class	63.0%
2nd Class	47.3%
3rd Class	24.2%

These figures show a clear stratification of survival by both sex and passenger class.

4.4 Machine Learning Model Performance

The Logistic Regression classifier achieved an accuracy of 81.6% on the held-out test set (179 passengers).

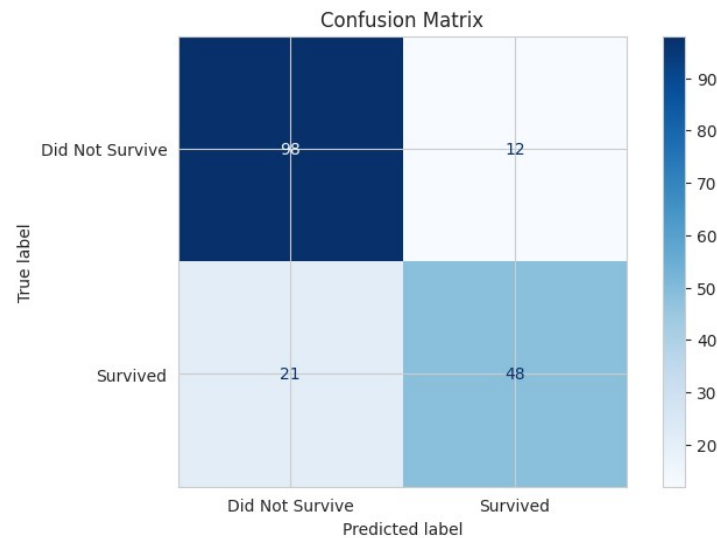


Figure 7: Confusion Matrix for the Logistic Regression Model

Metric	Did Not Survive (0)	Survived (1)
Precision	0.82	0.80
Recall	0.89	0.70
F1-score	0.86	0.74
Support	110	69

Overall accuracy across both classes was 82%, with a macro-averaged F1-score of 0.80 and a weighted-averaged F1-score of 0.81.

5. Discussion

The results confirm several well-documented patterns from the Titanic disaster. Sex was the single strongest determinant of survival, with women surviving at nearly four times the rate of men (74.2% versus 18.9%), consistent with an evacuation policy that prioritised women and children. Passenger class was also strongly associated with survival, with first-class passengers more than twice as likely to survive as third-class passengers (63.0% versus 24.2%), reflecting unequal access to lifeboats and to information during the evacuation.

The correlation analysis reinforced these patterns: Pclass and Fare were strongly negatively correlated, and Fare itself was positively correlated with Survived, indicating that the financial and social stratification of passengers translated directly into differences in survival outcomes. The Logistic Regression model, trained on seven predictors, achieved 81.6% accuracy, with noticeably stronger performance in identifying non-survivors (recall = 0.89) than survivors (recall = 0.70). This asymmetry is consistent with the class imbalance in the dataset (61.6% non-survivors versus 38.4% survivors) and with the fact that survival depended partly on circumstances — such as exact cabin location or timing during the evacuation — that are not captured by the available features.

Overall, the model coefficients and feature relationships both point to Sex and Pclass as the dominant predictors of survival, with Age, Fare, and family-size variables (SibSp, Parch) playing smaller supporting roles.

6. Conclusion

This project applied the full data science workflow — acquisition, cleaning, visualisation, statistical analysis, and machine learning — to the Titanic passenger dataset. The analysis showed that survival aboard the Titanic was strongly stratified by sex and passenger class, and a Logistic Regression classifier built on seven predictor variables was able to predict survival with approximately 81.6% accuracy on unseen data.

Limitations of the study include the relatively small sample size (891 of an estimated 2,224 people aboard), the need to impute a substantial proportion of missing Age values, the removal of the potentially informative Cabin

variable due to sparsity, and the use of a linear model that may not capture more complex, non-linear interactions between predictors.

Future work could explore extracting passenger titles from the Name field, engineering a family-size feature from SibSp and Parch, comparing Logistic Regression against non-linear models such as Random Forests or Gradient Boosting, and applying class-imbalance techniques to improve recall on the survivor class.

7. References

Kaggle. (n.d.). Titanic - Machine Learning from Disaster. Retrieved from <https://www.kaggle.com/competitions/titanic/data>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.

8. Appendix: Python Code

The full Python code used for this project (data acquisition, cleaning, visualisation, statistical analysis, and machine learning) is reproduced below. The complete, executable version is also submitted separately as a Jupyter Notebook (titanic_analysis.ipynb).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (8, 5)
%matplotlib inline

pd.set_option('display.max_columns', None)

# -----

# Load the dataset
df = pd.read_csv('titanic.csv')

# Dataset dimensions
print("Dataset dimensions (rows, columns):", df.shape)

# -----

# Column names
print("Column names:")
print(df.columns.tolist())

# -----

# First five observations
df.head()

# -----

# Data types
df.dtypes

# -----
```

```

# Detect missing values
missing_count = df.isnull().sum()
missing_pct = (missing_count / len(df) * 100).round(2)
missing_summary = pd.DataFrame({'Missing Count': missing_count,
                                'Missing %': missing_pct})
missing_summary[missing_summary['Missing Count'] > 0].sort_values('Missing Count',
ascending=False)

# -----

# Detect duplicated observations
duplicate_count = df.duplicated().sum()
print(f"Number of duplicated rows: {duplicate_count}")

# -----

# Handle missing values
df_clean = df.copy()

# Age: impute using the median age within each (Pclass, Sex) group.
# This is more informative than a single overall median because age
# strongly relates to passenger class and, to a lesser extent, sex.
df_clean['Age'] = df_clean.groupby(['Pclass', 'Sex'])['Age'].transform(
    lambda x: x.fillna(x.median())
)

# Cabin: drop the column entirely. Roughly 77% of values are missing,
# which is too sparse to impute reliably or to use as a predictor.
df_clean.drop(columns=['Cabin'], inplace=True)

# Embarked: only 2 missing values. Fill with the mode (most frequent
# port of embarkation) since the impact of this choice is negligible.
df_clean['Embarked'] = df_clean['Embarked'].fillna(df_clean['Embarked'].mode()[0])

# Remove duplicates, if any (none were found in this dataset)
df_clean = df_clean.drop_duplicates()

print("Missing values after cleaning:")
print(df_clean.isnull().sum())
print("\nShape after cleaning:", df_clean.shape)

# -----

plt.figure(figsize=(8, 5))
plt.hist(df_clean['Age'], bins=30, color='steelblue', edgecolor='black')
plt.title('Distribution of Passenger Ages')
plt.xlabel('Age (years)')
plt.ylabel('Frequency')
plt.show()

# -----

plt.figure(figsize=(6, 5))
sns.countplot(data=df_clean, x='Pclass', palette='viridis', hue='Pclass', legend=False)
plt.title('Passenger Class Distribution')
plt.xlabel('Passenger Class')
plt.ylabel('Number of Passengers')
plt.show()

# -----

plt.figure(figsize=(7, 5))
sns.boxplot(data=df_clean, x='Pclass', y='Age', hue='Pclass', palette='Set2', legend=False)
plt.title('Age Distribution by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Age (years)')
plt.show()

# -----

plt.figure(figsize=(7, 5))
sns.scatterplot(data=df_clean, x='Age', y='Fare', hue='Survived', alpha=0.7,
palette='coolwarm')
plt.title('Age vs Fare')
plt.xlabel('Age (years)')
plt.ylabel('Fare')
plt.legend(title='Survived')

```

```

plt.show()

# -----

plt.figure(figsize=(8, 6))
numeric_df = df_clean.select_dtypes(include=[np.number]).drop(columns=['PassengerId'])
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Variables')
plt.show()

# -----

pairplot_vars = ['Survived', 'Pclass', 'Age', 'Fare']
sns.pairplot(df_clean[pairplot_vars], hue='Survived', diag_kind='kde', palette='husl')
plt.suptitle('Pairplot of Selected Numerical Variables', y=1.02)
plt.show()

# -----

df_clean.describe(include='all').T

# -----

for col in ['Survived', 'Pclass', 'Sex', 'Embarked']:
    print(f"--- {col} ---")
    print(df_clean[col].value_counts())
    print(df_clean[col].value_counts(normalize=True).round(3))
    print()

# -----

numeric_df = df_clean.select_dtypes(include=[np.number]).drop(columns=['PassengerId'])
corr_matrix = numeric_df.corr()
corr_matrix

# -----

corr_pairs = corr_matrix.where(~np.eye(corr_matrix.shape[0], dtype=bool)).unstack().dropna()
corr_pairs = corr_pairs.sort_values(ascending=False)

print("Strongest positive correlation:")
print(corr_pairs.head(1))

print("\nStrongest negative correlation:")
print(corr_pairs.tail(1))

# -----

ml_df = df_clean.copy()

# Encode categorical variables
ml_df['Sex'] = ml_df['Sex'].map({'male': 0, 'female': 1})
ml_df = pd.get_dummies(ml_df, columns=['Embarked'], drop_first=True)

feature_cols = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked_Q', 'Embarked_S']
X = ml_df[feature_cols]
y = ml_df['Survived']

X.head()

# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

# -----

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

print("Model coefficients:")
for feature, coef in zip(feature_cols, model.coef_[0]):

```

```

    print(f" {feature}: {coef:.4f}")
# -----
y_pred = model.predict(X_test)
y_pred[:10]
# -----
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
# -----
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Did Not Survive',
'Survived'])
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix')
plt.show()
# -----
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=['Did Not Survive', 'Survived']))

```